

轮趣科技

K210 搭配云台例程 demo 使用说明

推荐关注我们的公众号获取更新资料



版本说明:

版本	日期	内容说明
V1.0	2024/07/05	第一次发布

网址: www.wheeltec.net

目录

1. 插入 SD 卡到 K210 并检查是否成功识别	1
2. 烧录云台控制板固件	4
3. 人脸跟随功能 demo	5
3.1 导入训练模型到 SD 卡	5
3.2 导入串口通信包到 SD 卡	5
3.3 将人脸跟随程序写入 SD 卡	5
3.4 接线说明	7
3.5 使用说明	8
4. 色块跟随功能 demo	9
4.1 将色块跟随程序写入 SD 卡	9
4.2 接线说明	9
4.3 使用说明	9
4.4 色块阈值修改教程	10
5. 色块学习并进行跟随 demo	12
5.1 写入源码到 SD 卡	12
5.2 接线说明	12
5.3 使用说明	12
6. 识别数字并发送到单片机 demo	14
6.1 写入源码到 SD 卡	14
6.2 接线说明	14
6.3 使用说明	14
7. 串口收发数据内容增删教程	15

1. 插入 SD 卡到 K210 并检查是否成功识别

在使用 K210 搭配云台的 demo 前，需要先将 SD 卡插入到 K210 中，并验证 K210 是否已成功识别到 SD 卡。请根据下列步骤确认：

- 1、插入 SD 卡到 K210 模块
- 2、将 K210 使用 typeC 连接到电脑的 USB 接口，打开“CanMV IDE”软件
- 3、点击左下角的连接图标，选择对应的 COM 口进行连接。COM 口号可以通过电脑设备管理器查询，芯片为 CH340.(若没有识别出 CH340, 请安装 CH340 驱动.)

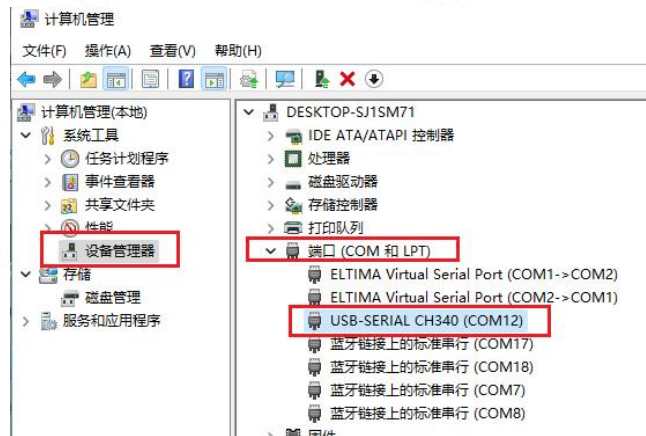


图 1-1 电脑设备管理器中识别 CH340 串口

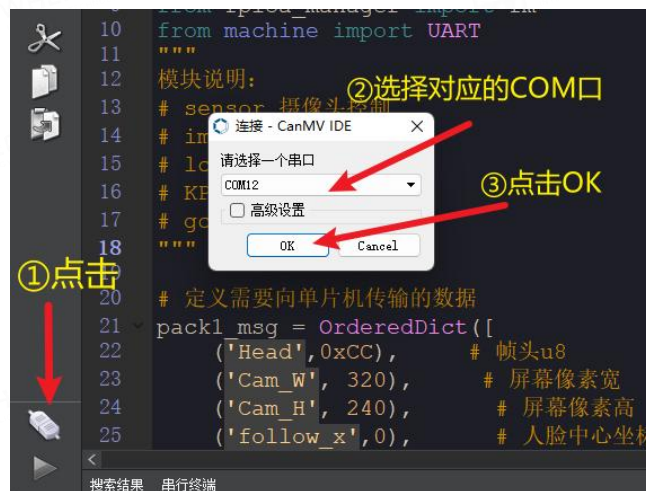


图 1-2 使用 CanMV IDE 连接 K210

- 4、在连接成功后，左下角图标变化如下：



图 1-3 连接成功示意图

5、找到文件“[./1.K210 例程 demo/main_TakePhotos.py](#)”并打开，接着点左下角绿色箭头运行。



图 1-4 打开测试文件

6、在程序运行后，按下 K210 上面的“BOOT”按键，拍摄一张照片。如果没有任何报错提示，则说明 SD 卡已经成功识别并可以正常保存数据。如果出现以下报错，则说明 SD 卡不存在或者未识别。



图 1-5 K210 BOOT 按键示意图

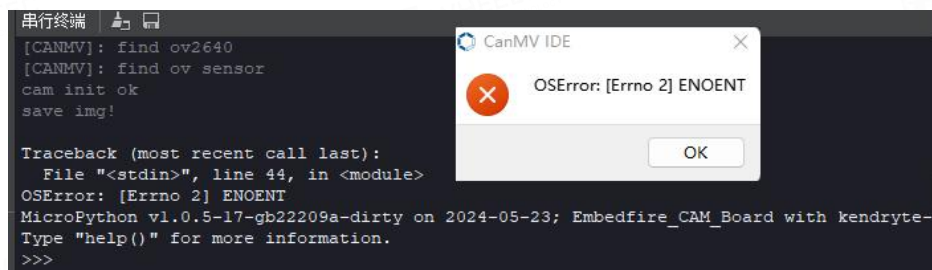


图 1-6 SD 卡未识别的报错信息

当 SD 卡无法识别时，请检查：

1、SD 卡是否已经正确插入

- 2、K210 断电后，重新插拔一下 SD 卡，再重新上电测试
- 3、将 SD 卡格式化为“FAT32”格式后，再重新断电插入测试
- 4、更换其他 SD 卡测试

注意：SD 卡不支持热插拔,请断电后再操作。K210 仅支持 FAT32 格式的 SD 卡。

2. 烧录云台控制板固件

烧录说明：默认的云台固件是不支持 K210 相关 demo 的，需要烧录本资料包中的云台固件才可以使用本教程中的所有 demo。

步骤①：使用 Type-C 连接 C06B 的串口 1，并接好下载使用的跳线帽。

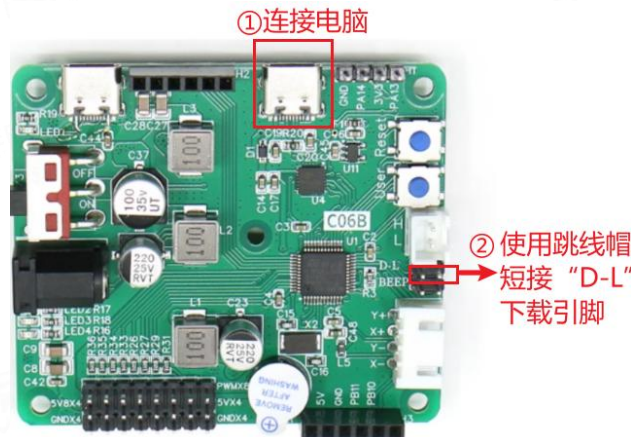


图 2-1 云台控制板烧录固件接线说明

步骤②：打开 Flymcu 软件，选择对应的 COM 口，在资料中找到“[./2.云台跟随 STM32 源码/云台跟随 STM32.hex](#)”文件进行烧录。注：若未识别 COM 号，请安装 CH9102 芯片驱动。

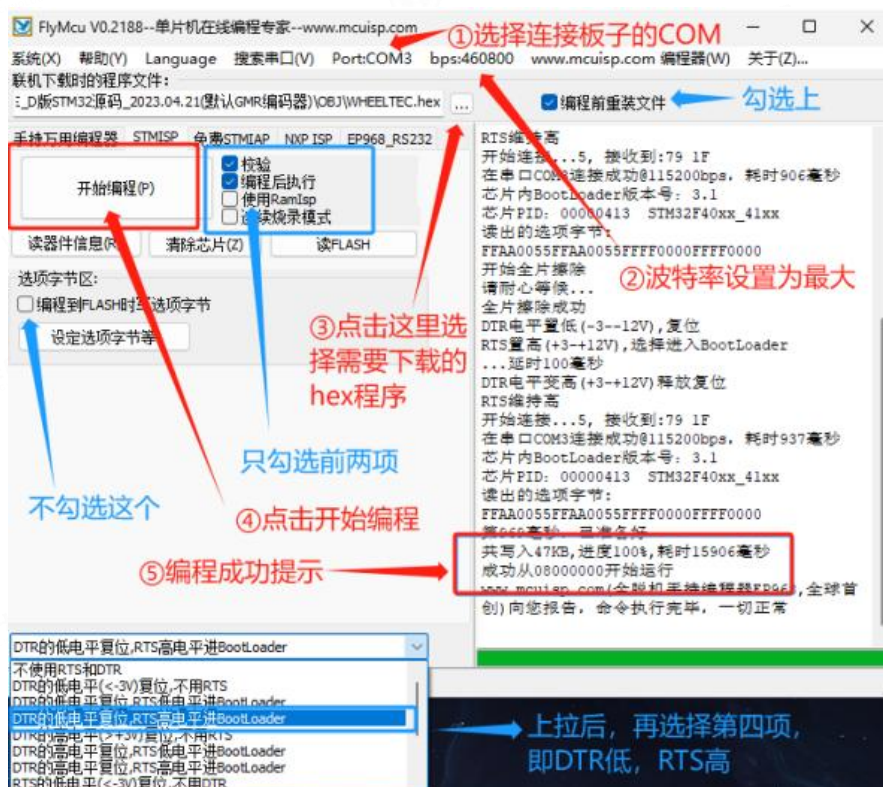


图 2-2 固件烧录配置说明

3. 人脸跟随功能 demo

3.1 导入训练模型到 SD 卡

在资料包中找到路径文件“[./3.野火 K210 AI 模型/KPU.zip](#)”。

野火K210 AI视觉相机 > 6-AI模型

KPU.zip

图 3-1 模型路径

将压缩包解压到 SD 卡中，注意需要带 KPU 文件夹，如下图所示：

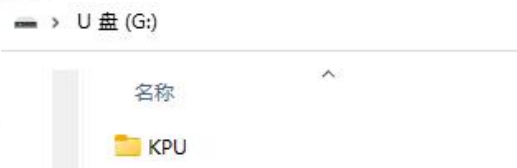


图 3-2 解压说明

解压完毕后，请将 K210 断电后再插入 SD 卡。

3.2 导入串口通信包到 SD 卡

在资料包中找到文件“[./1.K210 例程 demo/WHEELTEC_PackSerial.py](#)”，将其放置在 SD 卡根目录。如下图所示：

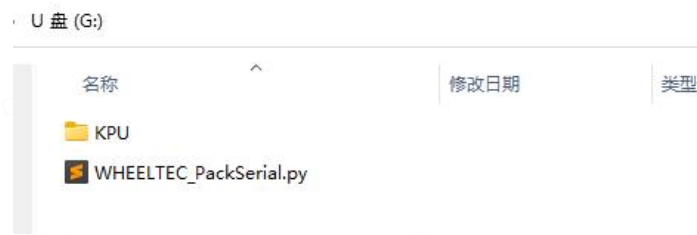


图 3-3 放入串口通信包

3.3 将人脸跟随程序写入 SD 卡

先参考[章节 1](#)，将 K210 设备连接到 CanMV Cam IDE.

找到资料中的文件“[./1.K210 例程 demo/main_Face_following.py](#)”并打开。

接着点击左下角的绿色开始按钮。测试程序是否可以正常运行并显示画面。如果程序运行后没有任何报错，则点击左下角的红色 X 按键停止运行，进行下一步。

在验证程序没有报错后，接着点击“工具”，选择“保存当前打开的脚本为

(main.py)到 CanMV Cam”，在弹出的窗口中选择“yes”，等待进度条完成即可写入成功，如下图所示。

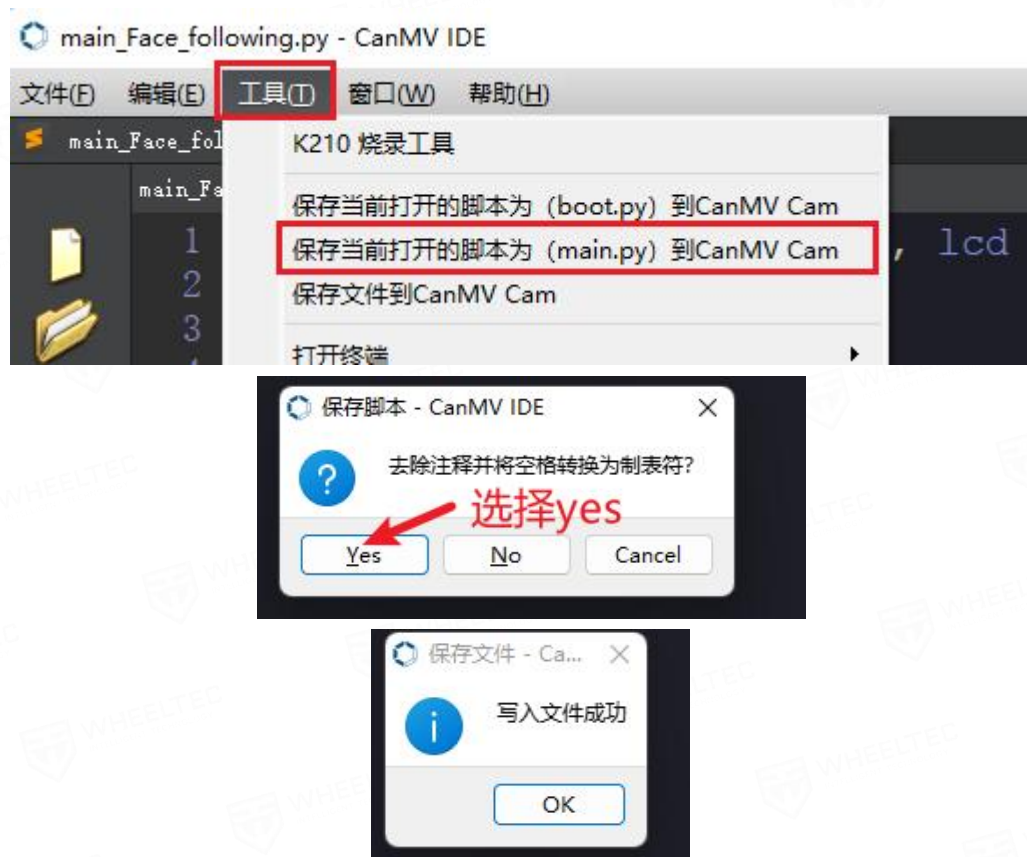


图 3-4 CanMV IDE 写入文件到 K210 图示

说明：在写入程序到 SD 卡后，当 K210 上电或复位时，会自动执行程序内容。需要注意的是，在写入程序前请确保 K210 已经正常识别 SD 卡，否则写入的内容将会被存放在 K210 的 Flash 芯片中，在 Flash 中缺少对应环境时，执行程序会出现报错。上述的程序依赖训练模型与通信包，故需要放在 SD 卡中执行。

3.4 接线说明

使用附赠的端子线连接 K210 的 4pin 端子和云台控制板的串口，引脚的编号如下图：

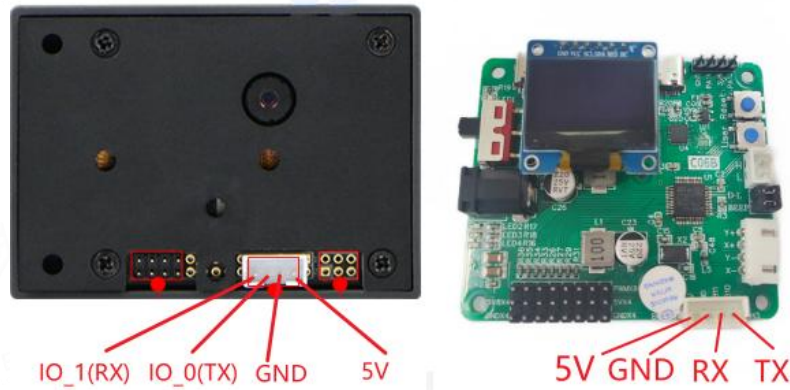


图 3-5 K210 与云台控制板引脚编号说明



图 3-6 K210 与云台控制板接线示意图

连接引脚对应说明：

K210	IO_1(RX)	IO_0(TX)	GND	5V
云台控制板(C06B)	PB10(TX)	PB11(TX)	GND	5V

注：在 K210 中，引脚是可以随意映射到各种功能的，在本例程中映射了 IO_1 作为串口 1 的 RX 引脚，IO_0 映射为串口 1 的 TX 引脚。

云台控制板与云台接线:

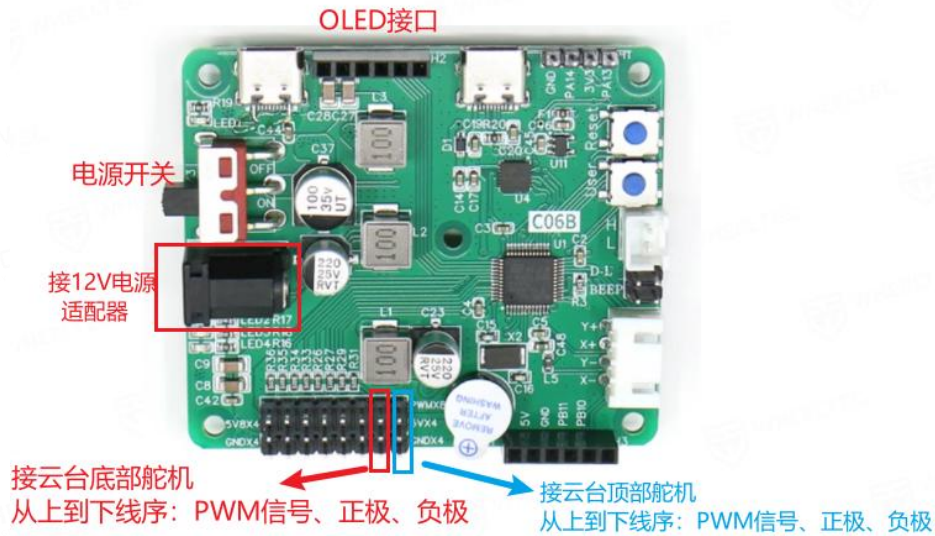


图 3-7 云台控制板与云台接线说明

3.5 使用说明

在接线完毕后，打开云台的电源开关。等待 K210 开机并运行程序后，即可开始进行人脸跟随。需要注意的是，受限于帧率，被跟随的人脸移动速度不能太快，否则会出现跟丢。当画面中出现多张人脸时，云台只会跟踪最大的人脸；当云台的姿态混乱无法纠正时，可以**长按**云台控制板上的“User”用户按键，此时云台会自动恢复到初始位置。

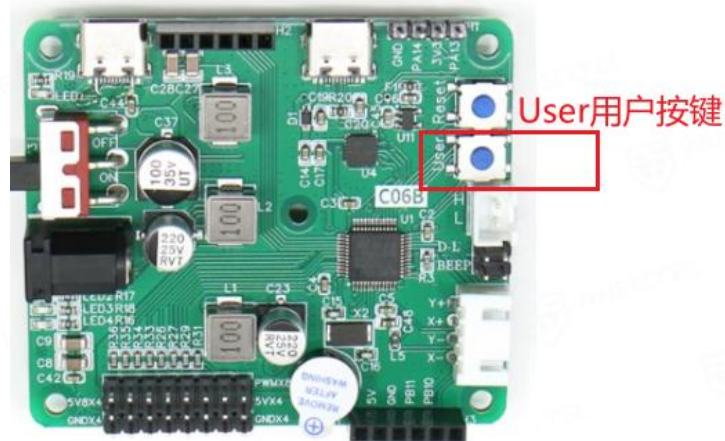


图 3-7 User 用户按键位置图

4. 色块跟随功能 demo

4.1 将色块跟随程序写入 SD 卡

先参考[章节 1](#)，将 K210 设备连接到 CanMV Cam IDE.

找到资料中的文件“[/1.K210 例程 demo/main_ColorBlock_following.py](#)”并打开。参考[章节 3.3](#)中的方法将程序写入到 SD 卡。

注意：此功能依赖串口通信包环境，如果未导入环境，请参考[章节 3.2](#)导入文件。

4.2 接线说明

接线与[章节 3.4](#)一样。

4.3 使用说明

在程序运行后，K210 的页面上最后一行会显示当前需要跟随的颜色。如果想切换跟随其他颜色的色块，可以通过单击云台控制板上的用户按键修改。内置了蓝、绿、黄、红四种颜色。同时，云台控制板上的 OLED 第一行也会显示当前跟随的是哪一种颜色的色块。

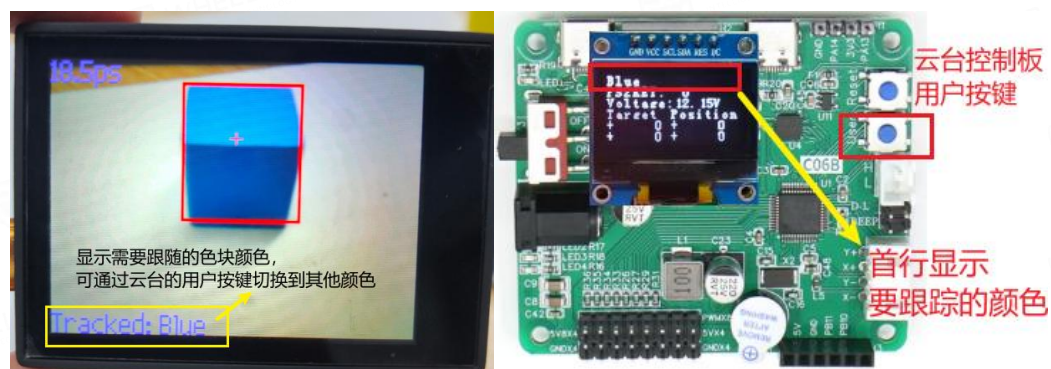


图 4-1 K210 显示与云台控制板图示

同样地，受限于帧率，被跟随的色块移动速度不能太快，否则会出现跟丢。当画面中出现多个色块时，云台只会跟踪最大的色块。当云台的姿态混乱无法纠正时，可以[长按](#)云台控制板上的“User”用户按键，此时云台会自动恢复到初始位置。

特别注意：色块跟随主要靠阈值来识别，当环境光与采集色块阈值时的光线

不一样时，会出现不能正常识别的情况，因此提供的源码里的阈值并不一定直接可用。这种情况需要自行采集自己环境中的色块照片，调节好阈值后修改代码并下载。下面将讲解如何修改色块阈值。

4.4 色块阈值修改教程

参考[章节1](#)，运行拍照程序“`./1.K210 例程 demo/main_TakePhotos.py`”。

我们将需要采集阈值的色块放置在需要跟随的环境中，通过 K210 上面的“BOOT” 按键进行拍照。可以多次按下按键拍摄多张照片，这些照片将会保存在 SD 卡上。拍摄完照片后，将 SD 卡接入电脑，可看到新拍摄的照片文件：

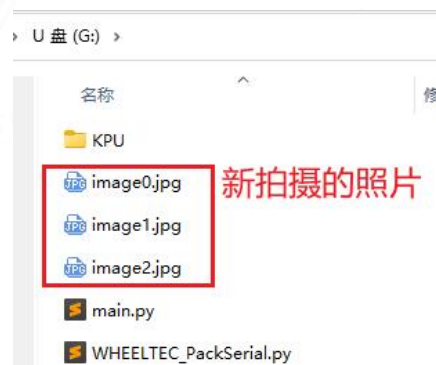


图 4-2 拍照后保存示意图

接着在“CanMV IDE”软件中，点击“工具”，选择“机器视觉”，选择“阈值编辑器”，点击“图像文件”，打开上一步拍摄的照片。

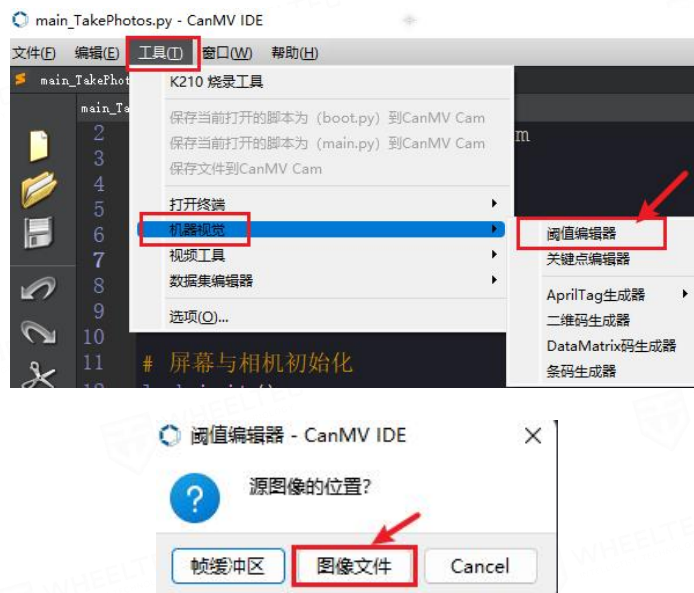


图 4-3 打开阈值编辑器

在打开照片后，我们通过滑动 6 个滑块，找到右边的画面只显示二进制图像

的色块的阈值，如下图所示。不同的环境光，不同的色块，对应的阈值不一样。在调节完毕后，把最后一行“LAB 阈值”记录保存。

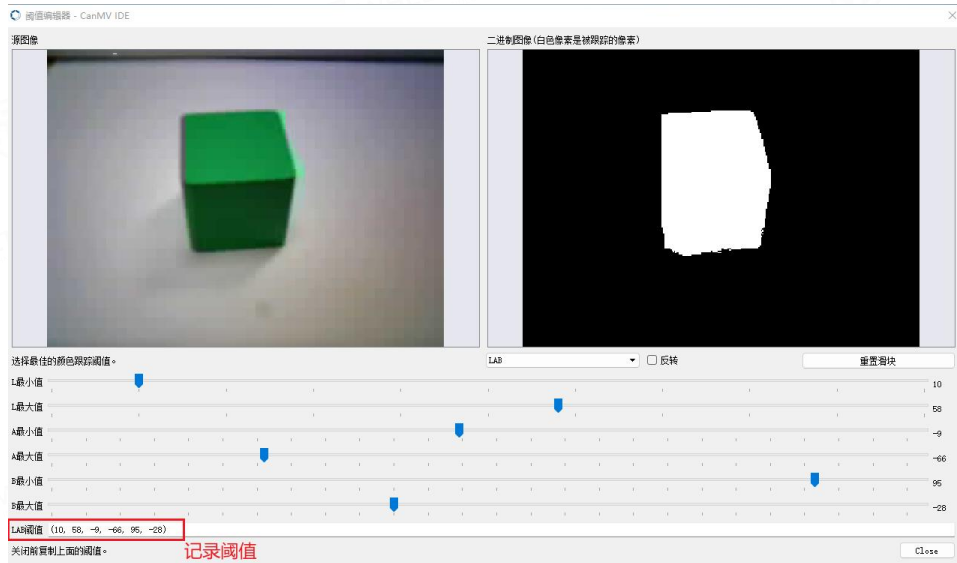


图 4-4 阈值编辑器调整示意

在设置好阈值后，重新打开文件“[main_ColorBlock_following.py](#)”。找到列表“color_thresholds”，将调整过的阈值对应更新到列表里即可。此处以绿色为例，将记录的阈值更新到绿色一栏。



图 4-5 更新代码里的阈值

在阈值更新完毕后，请参考[章节 3.3](#)中的方法重新将该程序程序写入到 SD 卡。

5. 色块学习并进行跟随 demo

5.1 写入源码到 SD 卡

先参考[章节 1](#)，将 K210 设备连接到 CanMV Cam IDE.

找到资料中的文件“[./1.K210 例程 demo/main_LearnColor_following.py](#)”并打开。参考[章节 3.3](#)中的方法将程序写入到 SD 卡。

注意：此功能依赖串口通信包环境，如果未导入环境，请参考[章节 3.2](#)导入文件。

5.2 接线说明

接线与[章节 3.4](#)一样。

5.3 使用说明

在程序运行后，K210 页面会显示一个白色的方框。将需要学习的色块放入白色框内，单击 K210 上的“BOOT”按键开始学习。在学习的过程中请保持相机、色块稳定。学习过程中出现的红色方框代表可识别的色块，同时学习进度在 K210 页面上会有显示。当进度到 100%时，代表学习完毕，开始跟随功能。



图 5-1 K210 学习色块功能示意图

注意：若学习完后识别有误，请复位 K210 后重新尝试。学习过程中，白色学习方框内不得有其他杂色。在学习完色块后，K210 屏幕底下会显示当前学习到的色块阈值，也可以将此阈值记下，更新到[章节 4](#)的源码中使用。

6. 识别数字并发送到单片机 demo

6.1 写入源码到 SD 卡

先参考[章节 1](#)，将 K210 设备连接到 CanMV Cam IDE.

找到资料中的文件“[./1.K210 例程 demo/main_DigitalRecognition.py](#)”并打开。

参考[章节 3.3](#)中的方法将程序写入到 SD 卡。

注意：此功能依赖串口通信包环境，如果未导入环境，请参考[章节 3.2](#)导入文件。

6.2 接线说明

接线与[章节 3.4](#)一样。

6.3 使用说明

将打印好的 0~9 任意一个数字放在 K210 相机前，K210 屏幕上会显示出识别结果，并将结果发送到云台控制板。在云台的控制板上的 OLED 第二行会显示接收到的结果值。

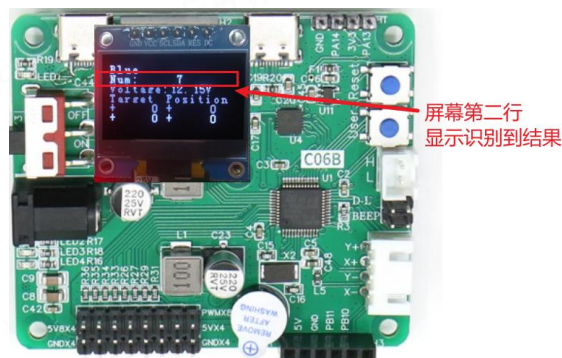
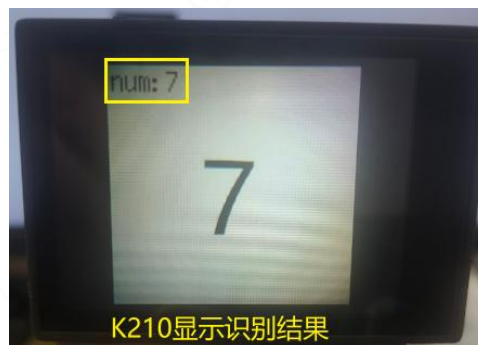


图 5-1 K210 识别数字示意图

7. 串口收发数据内容增删教程

若有修改串口数据需求，比如想新增发送一些标志位、其他复杂数据，或者减少一些数据的发送，可根据本章节教程修改。

以色块跟踪例程为例，打开文件 `main_ColorBlock_following.py`。比如现在需要新增发送一个 `float` 类型，变量名称为“`distacne`”的数据到单片机，数据可以在帧头“`Head`”和校验位“`BccCheck`”之间插入，如下图所示：

```
send_pack1_msg = OrderedDict([
    ('Head', 0xCC), # 帧头          uint8_t类型
    ('Cam_W', 320), # 相机的像素宽度 uint16_t 类型
    ('Cam_H', 240), # 相机的像素长度  uint16_t 类型
    ('follow_x', 0), # 需要跟踪的点x   uint16_t 类型
    ('follow_y', 0), # 需要跟踪的点y   uint16_t 类型
    ('distance', 0), # 新增需要发送的数据
    ('BccCheck', 0), # 数据BCC校验位  uint8_t类型
    ('End', 0xDD)   # 帧尾          uint8_t类型
])
```

图 7-1 字典“`send_pack1_msg`”修改

在新增该数据后，相应的需要在“`send_pack1_format`”数据类型上加入一个“`f`”代表新增 `float` 类型数据。

```
send_pack1_format = "<B4Hf2B"
# 加入数据类型
```

图 7-2 数据格式“`send_pack1_format`”修改

其中“`<`”代表小端模式，在 STM32 中数据默认是以小端模式存放。“`B`”代表第一个数据(Head)为 `uint8_t` 类型，`4H` 代表第 2 个数据开始后有 4 个连续的 `uint16_t` 类型数据(Cam_W,Cam_H,follow_x,follow_y)，`f` 代表我们新增的 `float` 类型数据(distance)，`2B` 代表后面有两个连续的 `uint8_t` 类型的数据(BccCheck,End)。

C 语言常用数据类型对应 python 里的字母如下表(未列出的自行上网搜查)：

<code>char / uint8_t</code> (1 字节无符号)	<code>B</code>
<code>Uint16_t</code> (2 字节无符号)	<code>H</code>
<code>Uint32_t</code> (4 字节无符号)	<code>I</code>
<code>Int8_t</code> (有符号 1 字节)	<code>b</code>
<code>Short / Int16_t</code> (有符号 2 字节)	<code>h</code>
<code>Int</code> (有符号 4 字节)	<code>i</code>

Float	(单精度浮点)	f
Double	(双精度浮点)	d

表 7-1 python 与 C 语言数据格式对应字母说明

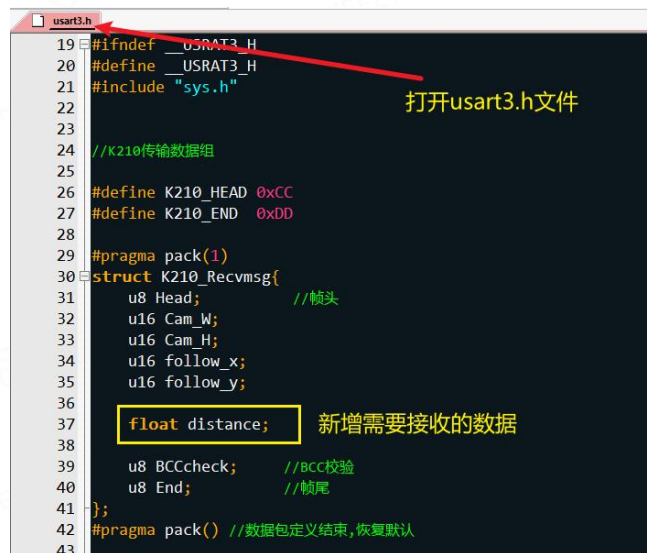
修改完上述两处地方后，接着修改数据赋值函数“update_sendpack1_data”，加入我们需要传输的数据即可。至此 python 文件修改的内容已全部完成。

```
# 更新需要发送的数据并返回打包结果,将结果直接发送到stm32端即可
def update_sendpack1_data(follow_x,follow_y,distance):
    global send_pack1
    send_pack1.msg['follow_x'] = follow_x          新增入口参数
    send_pack1.msg['follow_y'] = follow_y
    send_pack1.msg['distance'] = distance          新增赋值
    send_pack1.msg['BccCheck'] = send_pack1.pack_BCC_Value()
    return send_pack1.get_Pack_List()
""" 创建需要往stm32端发送数据的数据包 END """
```

图 7-3 赋值函数“update_sendpack1_data”修改

修改完 python 源码后，接着修改 stm32 源码。

打开“云台跟随 stm32 源码”，在工程内打开“usart3.h”文件，找到结构体“K210_Recvmsg”，并新增我们需要添加的内容“distance”，注意结构体里所有的数据类型都需要跟我们在 python 里定义的一样，如下所示：



```
19 #ifndef __USART3_H
20 #define __USART3_H
21 #include "sys.h"
22
23
24 //K210传输数据组
25
26 #define K210_HEAD 0xCC
27 #define K210_END 0xDD
28
29 #pragma pack(1)
30 struct K210_Recvmsg{
31     u8 Head;          //帧头
32     u16 Cam_W;
33     u16 Cam_H;
34     u16 follow_x;
35     u16 follow_y;
36
37     float distance;    新增需要接收的数据
38
39     u8 BCCcheck;       //BCC校验
40     u8 End;            //帧尾
41 };
42 #pragma pack() //数据包定义结束,恢复默认
43
```

图 7-4 STM32 接收端数据格式修改

在完成上述修改后，就可以正常使用数据了。由于数据是由 K210 发送到 STM32 的，所以数据的使用是在 STM32 端。打开“云台跟随 stm32 源码”工程，打开“usart3.c”文件，找到函数“K210_data_callback”，在数据解包完成后，可通过“k210_recv.distance”访问到我们新增的数据，如下图所示：

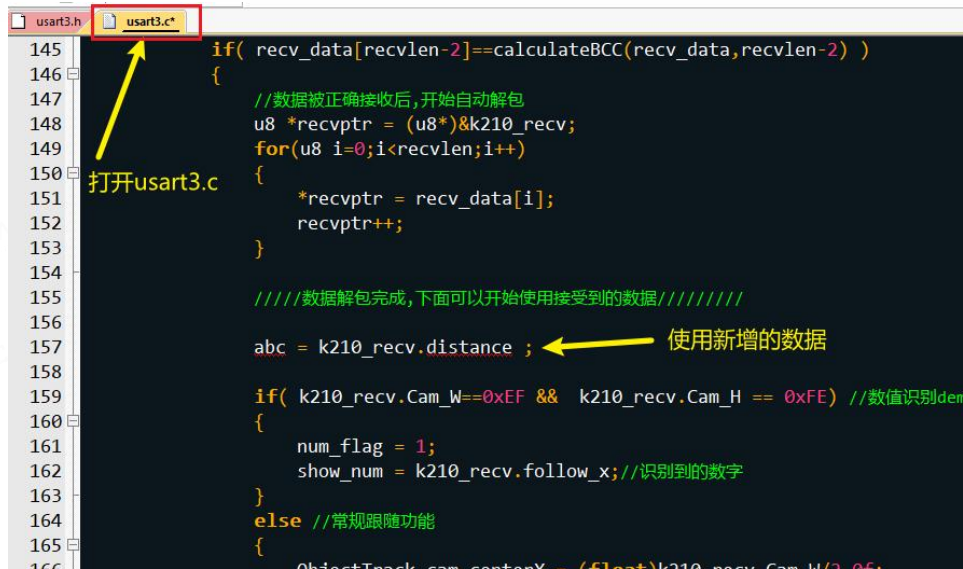


图 7-5 新增数据的使用

若需要修改发送的数据，也是同理。首先修改 python 源码加入或删除对应的数据，然后配置好数据类型；最后在 stm32 工程中对应的加入或删除数据即可。